

SUMMER RESEARCH INTERNSHIP PROGRAMME

2026

FINAL RESEARCH REPORT — SUBMISSION COVER PAGE

PROJECT / RESEARCH TITLE

Prosperity Score : An Alternative Credit Scoring Platform For Lending Informal Workforce

PROGRAMME DETAILS

Domain / Track: FinTech Duration: 15th May 2026 – 14th Aug 2026
Department: Computer Science Batch / Year: Batch 1 / 2026

SUBMITTED BY

Student 1 Krishnendu Ghosh
Student 2 Animikh Chowdhury

UNDER THE MENTORSHIP OF

Mentor Name Mr. Sajal Bhadra
Affiliation Consultant, TATA Consultancy Service Limited

DECLARATION

We hereby declare that this report is a record of genuine work carried out by us under the guidance of the above-mentioned mentor, as part of the IEEE SMC Student Branch Chapter, KGEC — Research Internship Programme 2026, and has not been submitted elsewhere for any other purpose.

Sajal Bhadra.

Mentor's Signature

Programme Coordinator

IEEE SMC Student Branch Chapter, KGEC | Research Internship Programme 2026 | Date of Submission: [14 / 08 / 2026]

Final Report of Internship Programme

Declaration

We, the below named student authors, declare hereby that AI-tools, like ChatGPT (OpenAI) and GitHub Copilot, were applied as supportive tools in the process of preparation and elaboration of the project report. In particular, ChatGPT helped in generating, checking, and modifying the code fragments, configuration examples, advising on debugging procedures, and writing explanations. GitHub Copilot provided support in the process of coding in the form of recommendations, minor corrections, and provision of information on the development process. The majority of AI assistance comprised the frontend works such as React/TypeScript components' implementation, including autofill functionality, employment matching process, touch-field control, and user-interface notifications. It also has the matter of the backend, where AI helped in implementing the systems with the help of FastAPI CORS configuration, secure cookie processing, serialization of responses, implementation of an API, and debugging the application. The AI-tools were also used for creating and modifying documentation, administrative materials, explanations for the inference logic of the project, and advice on the testing as well as deployment processes, including checking mechanisms and configuring environment variables for the launch.

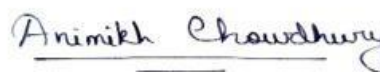
It has been determined that about 10% of the report has included elements of AI. Based on the policies of the Program, this is acceptable since the maximum share is 20% AI material. Among the 10% are approximately 6% of frontend and backend code modifications, 3% of documentation content, and 1% of minor word changes, proofreading, and edition. All of the elements of the project that were generated by AI were carefully analyzed, checked for accuracy, edited, and even tested by the authors, along with all of the components being incorporated into the project. Furthermore, even though the application was built and tested by the team, including all necessary aspects of frontend and backend work, API, CORS, and autofilling, everything generated by AI was accepted only after being looked at by the authors. Therefore, the group is responsible for the results of the project and ensures that all AI elements used during the completion of the assignment have been secondary.

Signatures We confirm the accuracy of this declaration and that the AI usage described above reflects the assistance received during preparation of this report.



Student 1: Krishnendu Ghosh

Date- 12/08/2026



Student 2: Animikh Chowdhury

Date-12/08/2026

Final Report of Internship Programme

1. Introduction:

The availability of traditional credit is a significant element in the financial planning of individuals and businesses. However, traditional credit scoring systems rely on already established credit histories, making them adverse to people with little or no traditional credit history. Gig workers, the self-employed, small business owners, and first-time borrowers might have very limited representation in conventional credit systems, even though they have a steady income and responsible financial behavior. At the same time, bank statements can provide plenty of information about someone's financial activities, including income, expenses, cash flows, repayment behavior, and spending habits.

ProsperityScore is aimed at filling that gap. It suggests an alternative credit scoring algorithm that uses data from bank transactions. The new system receives a bank statement from the user, processes all the transactions, sorts them into existing categories, and extracts necessary insights about income, expenses, cash flows, repayment history, and behavior for scoring the person.

2. Objective:

This is the project that is planned and here is the way this project must be implemented. The objective of the project is to create a new credit assessment method where the assessment is based on real behavior of a person presented in bank statements.

To achieve this goal, there are the following small tasks for the project:

- Make it possible to receive and process various bank statement types (in CSV, XLS or XLSX formats) without depending on the layout provided by a particular bank.
- Identify and standardize data on date, amount, description, payee, and balance for creating one set of common data on each transaction.
- Classify transactions into the following types: income, fixed expenses, variable payments, discretionary expenses, transfers, repayments, refusals.
- Calculate ratios that show the stability of income, burden of expenses, stability of cash flow, attitude toward repayments, and financial behavior.
- Combine all calculated metrics in one score from 0 to 100 that corresponds to ProsperityScore from 300 to 900 with the description of the reason for the score.
- Separate risk classification from eligibility thresholds in order to be able to extract any information on affordability or repayability.
- Provide users with information about the estimated emi amount that can be paid by him/her , approximate loan amount, and human readable reasons supporting the result.
- Deliver the assessment through a guided web workflow while keeping scoring logic centralized in the backend.

3.Literature and Background

Traditional credit scoring determines an individual's creditworthiness primarily using historical financial records and previous credit behaviour. Alternative credit scoring extends this approach by incorporating additional transactional and behavioural information alongside conventional credit data. This approach is particularly beneficial for applicants with limited or insufficient traditional credit histories. Bank-statement data is valuable in this context because it provides direct evidence of an individual's actual income inflows, expenditure patterns, account balances, and repayment activities.

Final Report of Internship Programme

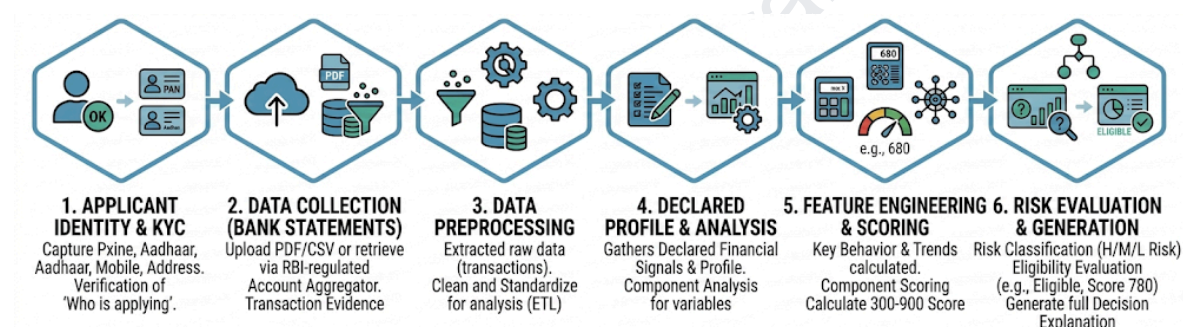
A major challenge in alternative credit assessment is the conversion of semi-structured transaction data into reliable financial features. Bank statements may contain bank-specific terminology, abbreviations, merchant names, and varying transaction descriptions, causing similar financial activities to be represented differently across institutions. Therefore, an effective preprocessing and normalization layer is required before transaction categorization and feature aggregation can be performed reliably.

Transparency is another important consideration in credit assessment, as a numerical score alone does not explain the factors that influenced the final decision. ProsperityScore addresses this issue by using individual component scores and an explainability mechanism to identify significant positive and negative financial indicators. These indicators may include stable income, a high expense-to-income ratio, frequent bounced transactions, negative account balances, low savings behaviour, and repayment consistency.

The transaction-processing architecture combines deterministic rule-based techniques with an optional machine-learning component. The current implementation primarily uses keyword matching and regular expressions to identify transaction categories, while a TF-IDF-based representation combined with Logistic Regression can be used for text-based transaction classification. This hybrid architecture enables reliable handling of known transaction patterns through deterministic rules while also providing the capability to generalize from labelled transaction data.

4. Methodology:

The complete system workflow is:



The architecture contains two main parts. The React/TypeScript frontend collects applicant information, accepts the bank statement, supports review and consent, and presents the final result. The Python/FastAPI backend performs the financial analysis, scoring and decision logic. The frontend does not calculate the financial score; the backend remains the authoritative scoring layer.

Layer	Technology
Frontend	React , TypeScript, Tailwind CSS
Backend	Python , FastAPI
Data processing	Pandas, Numpy, Scikit-learn (LinearRegression), openpyxl
Transaction categorization	Rule-based regex engine (primary) with an optional joblib-persisted ML classifier
Deployment	Render , Vercel (Annexure 1)

Final Report of Internship Programme

4.1 User Input and Statement Processing:

The application follows a sequential workflow consisting of Identity → Profile → Statement → Review → Result. The Identity & KYC stage collects essential applicant information such as full name, business or occupation, PAN, Aadhaar, mobile number, email, city, and PIN code. The applicant enters the required details and proceeds to the next stage using the Continue option (Fig 1).

Fig 1: Identity Stage

The bank statement is the primary scoring input. In the current implementation, supported formats are CSV, XLS and XLSX. The frontend sends the file to the backend using multipart form-data through POST /score. PDF/OCR is described as a possible broader architecture but is not part of the documented current CSV/XLS/XLSX implementation (Fig 2).

Fig 2: BankStatement upload Interface

4.2.Data Cleaning and Normalization:

The loader is flexible with respect to bank formats. It identifies potential date and amount columns by analyzing the name of each column and confirming the values that populate it. Common names of date columns are Date,

Final Report of Internship Programme

Transaction Date, Value Date, and Posting Date. The loader can accommodate an Amount field that can be either a single signed value or separate Credit and Debit fields.

$$\text{Amount} = \text{Credit} - \text{Debit}$$

Whenever Excel serial dates are encountered, they are being interpreted. Rows that could not be interpreted are finally purged from the dataset.

4.3. Transaction Categorization:

Categorization is required because subsequent financial calculations depend on the meaning of each transaction. The seven core categories are Income, Fixed Expenses, Variable Expenses, Discretionary Expenses, Transfers, Loan Repayments and Bounces.

The primary rule-based engine uses transaction-type information where available and keyword/regular-expression matching on descriptions and payee information. Terms such as salary, payroll, freelance and stipend can indicate Income; EMI, loan repayment, home loan, personal loan and credit-card payment can indicate Loan Repayment; and bounce, failed debit, dishonour, insufficient funds, NACH fail and ECS return can indicate Bounces.

The ML layer uses TF-IDF to convert transaction descriptions into numerical features and Logistic Regression to classify labelled transaction categories. The rule-based layer can act as a fallback when the classifier is unavailable or unreliable.

Fig 3 : Financial Profile of the User

4.4. Feature Engineering:

Once categorized, transactions get pooled into five essential pillars:

- **Income:** the average income per month, reliability of income and its sources. Its stability (S) is calculated as $1 - (\sigma/x)$. The formula divides metrics into a 30% value of average income, 40% of stability, 15% of source diversity, and 15% of growth.
- **Expense:** the proportion of expenses vs income along with trends of expenses. The ratio is calculated as $\text{EIR} = \text{Total Expenses} / \text{Total Income}$, wherein an EIR of <50% is defined as Excellent, >50% but <70% as Good, >70% but <90% as Risky, and $\geq 90\%$ as High Risk.

Final Report of Internship Programme

- Cashflow: the average account balance, the minimum balance, the occurrence of negative balances, and the trend of balances. In case the running balance is not available, the balances can be recreated based on the accumulated amounts of transactions.
- Repayment: repayment amounts of PMT and loans, on-time repayment rate, bounce rate and credit-card payment behaviour. The bounce rate is calculated using $\text{Bounce Rate} = \text{Failed Transactions} / \text{Total Debits}$.
- Behaviour: savings and expenditures, expenditure stability, and amounts of money at the end of the month. Behaviour is used as a secondary explanation signal in the super equation

4.5. Composite Score and Risk

The Final Score can be calculated by using the formula :

$$\text{Final Score} = (0.30 * \text{Income}) + (0.20 * \text{Expense}) + (0.20 * \text{Cashflow}) + (0.30 * \text{Repayment}). \text{ (Fig 4)}$$

The primary elements are assigned values from 0–100. The higher rankings of Income and Repayment are attributed to the importance of income sustainability

Component	Weight
Income	30%
Expense	20%
Cashflow	20%
Repayment	30%

Fig 4: Primary component weighting

Final Score	Risk Category
≥ 80	Low Risk
60–79	Medium Risk
40–59	High Risk
< 40	Very High Risk

Fig 5: Risk classification reference.

$$\text{ProsperityScore} = 300 + (\text{FinalScore} * 6)$$

4.6 Eligibility, EMI and Explainability:

The risk classification differs from the loan approval method. The established categories are ≥ 80 Low Risk, 60–79 Medium Risk, 40–59 High Risk, and < 40 Very High Risk. The eligibility requirements might instantly deny or highlight the applicants even if they have received a high score.

The eligible EMI shall be 30% of the average salary.

$$\text{Eligible EMI} = 30\% \times \text{Average Monthly Income}$$

$$\text{Approximate Maximum Loan} = \text{Eligible EMI} \times 36$$

Thus, the estimated maximum loan is equal to eligible EMI times 36.

Final Report of Internship Programme

Suppose the monthly salary is ₹ 50,000, then the eligible EMI becomes ₹ 15,000, while the total amount would be ₹ 5,40,000 for the easiest 36-month loan.

This comes to a mere estimate of affordability rather than amortization.

The explanation engine detects signals crucial for the output. Among negative signals, high bounce rate, low repayment rate, negative balance, poor income stability, high EIR, low amount in savings, and many simultaneously paid EMIs can be mentioned. In contrast, steady salary, stable cash flows, and high level of repayment stability are positive signals.

4.7. Frontend Workflow, Review and Consent:

Communication changes occur from the frontend to backend using REST APIs. The sequence is Identity → Profile → Statement → Review → Result. The /score endpoint returns individual component scores, the final score in the range of 300-900, risk classification, eligibility determination, eligible EMI, approximate loan amount, and explanation of the evaluation process.

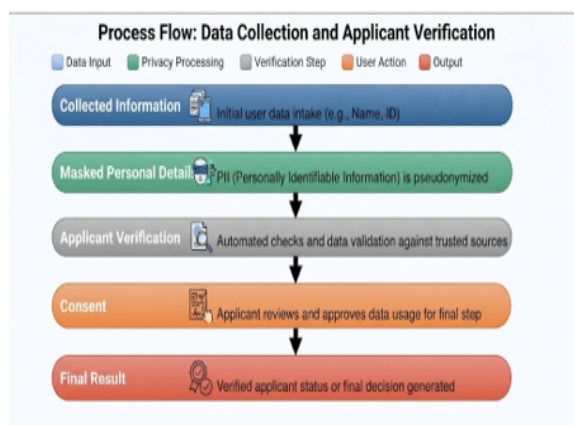


Fig 6 : Review Stage

Prosperity Score
Alternative Credit Ledger

Turn a bank statement into a 300-900 credit score

Just in Four steps: Identity, statement, declared profile, review with a full explanation of the decision.

1 Identity & KYC (Who is applying)
2 Bank statement (Transaction evidence)
3 Financial profile (Declared signals)
4 Review & generate (Consent and score)

Review & generate

Masked summary of captured data, then score generation:

FULL NAME Arunabh Chowdhury	EMPLOYMENT TYPE Business owner	PAN P*****7	AADHAAR *****3641
MOBILE +91 8779388379	EMAIL arunabhchowdhury2003@gmail.com	CITY / PIN Kolkata 700054	MONTHLY INCOME ₹99,641
AVG. BANK BALANCE ₹1,06,18	MONTHLY RENT 0 (None)	EXISTING EMI 0 (None)	OTHER OPEN LOANS 0 Loans
UTILITY BILLS ON TIME No	FINANCIAL Arunabh_Bank_statements.csv		

☒ I/We consent: the applicant permits processing of PII and Aadhaar details for the purpose of generating this Prosperity Score.

Generate score

Fig 7: Review and Consent Interface

Sensitive information such as PAN and Aadhaar is masked or redacted on the review screen. The user verifies the collected and extracted information and provides consent before the final assessment is presented (Fig 7).

Final Report of Internship Programme

5. Results & Discussion

The ProsperityScore technology is an innovative solution for alternative credit assessment that enables to analyze the data from bank statements and provide a relevant financial evaluation based on a set of factors. The technology offers a complete set of processes, ranging from the submission of bank statements to calculating the risk level and eligibility. Income, Expenses, Cashflow, Repayment, and Behavior scores are assessed to provide an overview of the applicant's financial position based on the 300-900 scale (Fig 8).

The results of the technology testing, conducted with the transaction data samples, below, suggest that each stage of the technology has been implemented in an appropriate manner. The tested application has received a score of 86.48 or 819 points out of 900, indicating a Low Risk level. It can be concluded that the technology has the ability to transform data into relevant information.

To summarize, it has been found that the behavior reflected in bank statements can be incorporated in the credit assessment process as it provides an opportunity to increase reliability and transparency of the assessment.

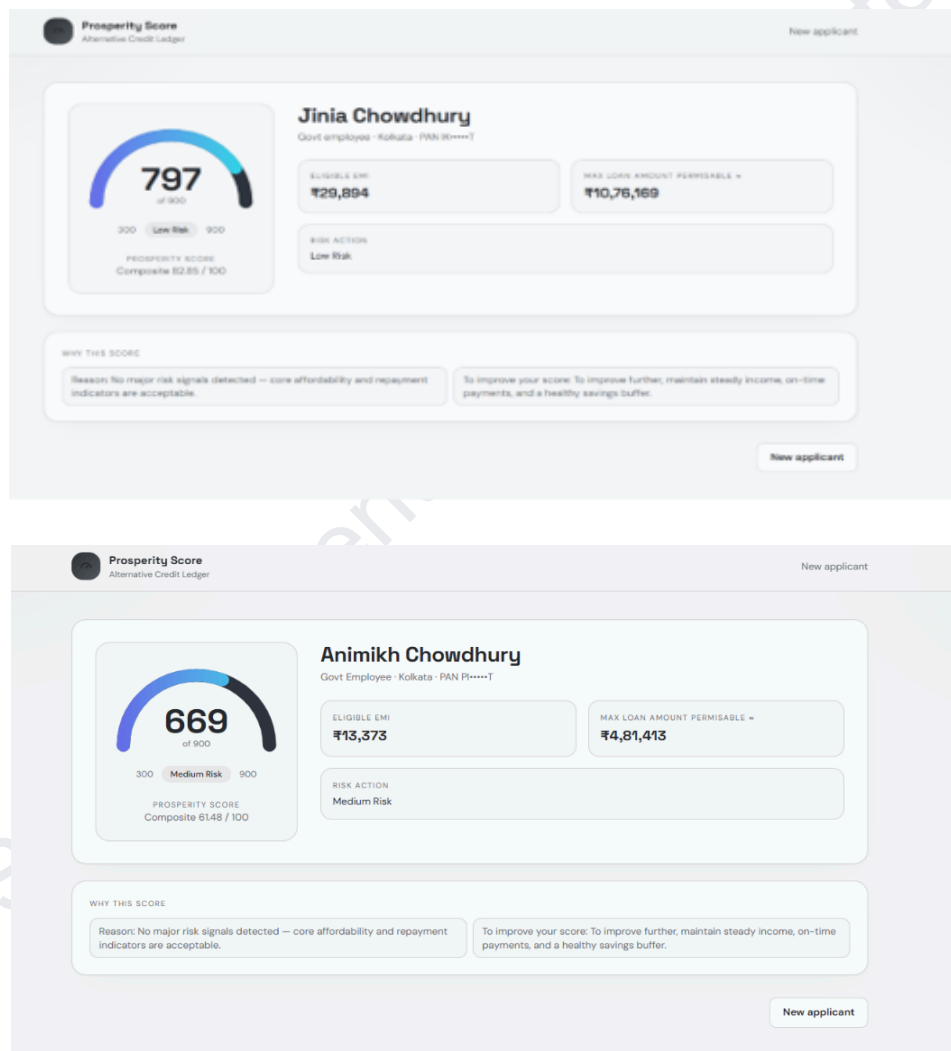


Fig 8. Final Prosperity Score result screen.

Final Report of Internship Programme

6. Conclusion

Prosperity Score proposes a realistic answer to credit scoring by evaluating activity in bank statements in order to produce a clear presentation of one's financial situation. This solution solves the core technical challenges involved with this approach as it includes collecting bills in different types, processing, classifying transactions, building features, calculating scores, finding out risk ability, finding suitability and giving explanations.

The main score combines Income, Expense, Cashflow and Repayment with particular weights calculated with a help of some system resulting in a score that varies from 300 to 900. This score is calculated with the help of some behavior signals but it is important to distinguish between risk classification and suitability criteria requirements to ensure that the score is not used as an indicator for credit approval.

The complete stack implementation provides a clear distinction between the tasks of React/TypeScript frontend and Python/FastAPI backend.

In the future, RBI Account Aggregator (AA) framework capabilities might be adapted that help collect financial data using a consent-based mechanism rather than relying on bank statements. With such integration, users can share sensitive financial data with their financial providers in a secured manner by employing an Account Aggregator, thus avoiding the need for using any paper documents for data collection.

In order to improve Prosper Financial's product, it is necessary to create a more comprehensive labeled data set, ensure that it is validated against real repayment data, implement better predictive tools, check for fairness and discrimination, upgrade transaction classification, provide PDF/OCR features, and enhance security and reliability features, as well as to implement RBI aggregator integration and ensure the proper operation of the loan amortization engine. The result of these enhancements will be a better reputation of ProsperityScore as an alternative system for credit assessment.

PROJECT ACCESS AND RESOURCE LINKS

The Prosperity Score: An Alternative Credit Scoring Platform for Bank Statement Analysis has been implemented as a full-stack web application with a React/TypeScript frontend and Python/ FastAPI backend. The following resources provide access to the deployed application, source code, and backend API for demonstration and technical evaluation.

A. Live Application

Prosperity Score – Web Application

The deployed web application provides the complete workflow for applicant information entry, bank-statement upload, financial analysis, review and consent, and generation of the final Prosperity Score.

Deployment Link: <https://prosperityscore.vercel.app/>

B. Source Code Repository

The GitHub repository contains the source code and implementation resources for the frontend, backend, transaction processing, scoring logic, configuration, and deployment.

GitHub Repository:

Frontend <https://github.com/techkrishnendu29/Prosperity-Frontend>

Backend API <https://github.com/techkrishnendu29/Purchase-Capability-IEEE-KGEC>

C. Backend API

Prosperity Score – FastAPI Backend

The backend provides the core financial-analysis and scoring services. It processes supported bank-statement files, categorizes transactions, extracts financial features, calculates component scores, determines risk and eligibility, and generates the final assessment.

API Base URL: <https://purchase-capability-ieee-kgec.onrender.com/>

Primary Endpoint:

POST /score

POST/statement/summary

The /score endpoint receives the bank statement through multipart form-data and returns component-wise scores, the final 300–900 Prosperity Score, risk classification, eligibility information, eligible EMI, approximate loan amount, and explainable assessment results.

Note: The deployed architecture uses Vercel for the frontend and Render for backend deployment, as documented in the project report. These resources collectively provide access to the deployed system, source code, backend services, and API interface, enabling independent demonstration and technical evaluation of the Prosperity Score platform.